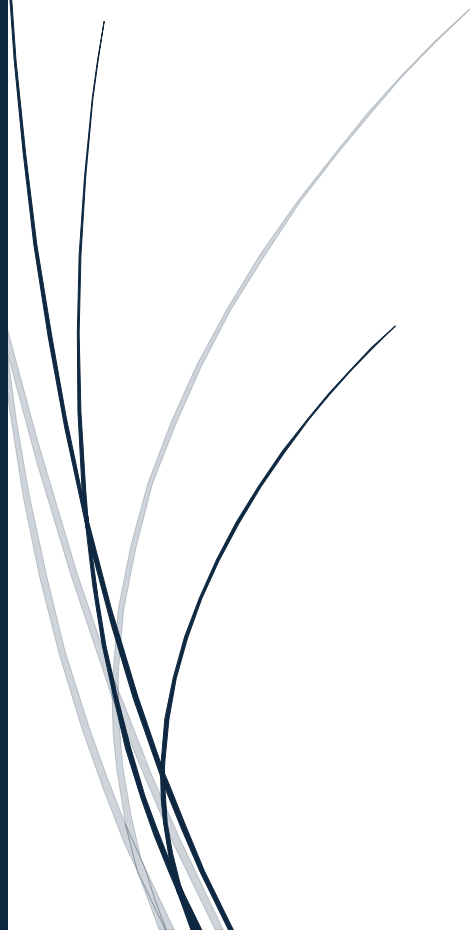




2/5/2026

CS1813 SOFTWARE DESIGN

ASSIGNMENT 2



Albert Guo
BRUNEL UNIVERSITY

Contents

1.SOFTWARE REQUIREMENTS SPECIFICATION	2
1a) GENERATIVE AI/LLM USED	2
1b) REQUIREMENTS SPECIFICATION GENERATED BY THE LLM	2
1c) IMPROVED SOFTWARE REQUIREMENTS SPECIFICATION	5
2. ALGORITHM DESIGN.....	10
2a) FLOWCHART GENERATED BY LLM.....	11
2b) LLM-GENERATED FLOWCHART CRITIQUE	12
2c) HOW I IMPROVED ON THE LLM-GENERATED FLOWCHART	12
2d) Improved main flowchart	13
2.1d) INITIALIZATION AND REGISTRATION.....	14
2.2d) GENERATE & ROLL DICE	15
2.3d) USER’S TURN.....	16
2.4d) SWIFTBOT’S TURN	17
2.5d) CHECK WIN/DRAW CONDITIONS & BLINK LED.....	17
2.6d) MAINTAIN LOG & UPDATE SCOREBOARD	18
2.7d) PLAY AGAIN/QUIT OPTIONS	18
3. USER INTERFACE DESIGN.....	19
3a) LLM-generated UI designs	19
3b) Improved UI prototype designs	22
3c) HOW I IMPROVED ON THE EXISTING GENERATED UI DESIGNS	26
4) PLANNING AND MONITORING PROGRESS	26
4a) GANTT CHART FOR TRACKING/MONITORING PROGRESS & PLANNING FOR SOFTWARE IMPLEMENTATION IN ASSIGNMENT 2 & 3.....	27
5) REFERENCES.....	28

SOFTWARE REQUIREMENTS SPECIFICATION

GENERATIVE AI/LLM USED

LLM used: OpenAI ChatGPT

Model/version used: GPT-5.1 mini

Access method: ChatGPT web (free tier)

Date & time accessed to generate the SRS: 2025-12-13 15:30 UTC (approx.)

Note: ChatGPT doesn't provide per-message timestamps, and I can't view the first time the message was sent in the Microsoft edge's search history, as it automatically updates the date accessed when I accessed the chat again, I would have to give an accurate approximation of the most likely dates and times based on the last time I used the LLM to guide me with my SRS.

Prompts used

- 1) **to generate the srs:** *"Can you generate the starting sections of a report with the title and contents and only after that generate the functional, non-functional and additional functionalities. Ensure that these follow the standards of the example srs (the example with the title highlighted in green, not yellow) and only follow the contents of the srs that is outlined in the assignment 2 brief. Leave the rest of the sections like algorithm design alone."*
- 2) **To make the srs clear, structured and numbered:** *"Can you make the functional, non-functional and additional functionalities written as numbered points to make it clear and structured of each key point"*
- 3) **to help me structure the LLM subsection:** *"how should I structure the LLM subsection in the srs section where you state the LLM used, model version, date your accessed it and the prompts you used AND how do I say the description of the improvements i made to the generated LLM srs in a structured and what way because I'm confused in how should I present it"*

REQUIREMENTS SPECIFICATION GENERATED BY THE LLM

Functional Requirements

- 1) **FR-01. (System)** The system shall display a welcome message and prompt the user to start the program by pressing SwiftBot **button 'A'**.
Acceptance: When button A is pressed the CLI clears the welcome and proceeds to registration.
- 2) **FR-02. (System / User)** The system shall prompt the user (via CLI) to enter their preferred player name using the keyboard. The user shall enter an alphanumeric name (no leading/trailing whitespace).
Acceptance: Name accepted only if it matches regex `[A-Za-z0-9]{1,30}`; otherwise show friendly error and re-prompt.
- 3) **FR-03. (System)** The system shall assign a name to the SwiftBot automatically (random or configured). The user may optionally override this name via a CLI option before the first round.
Acceptance: SwiftBot name appears in move summaries and logs.

- 4) **FR-04. (System)** The system shall roll a virtual 6-sided die for both players to determine who goes first. The player with the higher roll goes first and is assigned 'O' (noughts); the other is assigned 'X' (crosses). If a tie occurs, the system shall re-roll until tie resolved.
Acceptance: Die roll values displayed, first player and piece assignment displayed.
- 5) **FR-05. (User/System)** On a human turn the user shall specify a move in the format [row, column] where row and column are integers 1-3. The system shall parse and validate the input.
Acceptance: Valid input accepted; invalid input triggers clear error message and re-prompt (no crash).
- 6) **FR-06. (System)** The system shall prevent placement on an already occupied square. If user attempts that, the system shall inform the user and ask for a different move.
Acceptance: No move overwrites existing piece; log records the attempted invalid move.
- 7) **FR-07. (System)** During the SwiftBot's turn the system shall select the SwiftBot move (random by default) and announce the chosen square to the user in the CLI before the robot moves.
Acceptance: CLI shows e.g. [SwiftBot – X] will move to square [2,1] then SwiftBot moves.
- 8) **FR-08. (System → SwiftBot)** After selecting its move, the System shall command the SwiftBot to physically travel to the target square, blink green 3 times, and return to its starting position on the board. The System is responsible for sending motion commands; SwiftBot executes them.
Acceptance: Motion command logged; LED blink sequence performed. (Motion accuracy is verified by calibration tests recorded in Implementation.)
- 9) **FR-09. (System)** After every move (user or SwiftBot) the System shall update the CLI ASCII board and print a concise move summary: [Yasoda – O] moved to square [1,3].
Acceptance: ASCII board and a one-line summary displayed within 1–2 seconds of confirming the move.
- 10) **FR-10. (System)** The system shall detect win or draw conditions after each move. For a winning line the System shall command the SwiftBot to trace the winning line and perform the appropriate LED effect: — **User (O) win:** blink green 3 times before and after tracing; — **SwiftBot (X) win:** blink red 3 times before and after tracing. For a draw, the SwiftBot shall spin once and blink blue 3 times.
Acceptance: Correct outcome detection and corresponding SwiftBot behaviour executed and logged.
- 11) **FR-11. (System)** After each round the System shall append a round entry to a persistent log file containing round number, each move in order (player, piece, row, column), winner/draw, winning player name (or draw), and timestamp. The log file path shall be shown to the user on program termination.
Acceptance: Log file contains human-readable CSV or structured text entries.
- 12) **FR-12. (System)** The System shall maintain a scoreboard across rounds (persisted during the program execution; persisted to disk on termination). A round win increments that player's score by 1. The scoreboard shall be displayed after each round.
Acceptance: Scoreboard persists for the program session and is shown at end of each round.
- 13) **FR-13. (User/System)** At the end of a round the user shall be prompted to continue (press SwiftBot 'Y') or quit (press SwiftBot 'X'). If user quits the program, the system

writes final logs, displays log path, and terminates cleanly.

Acceptance: On quit the program writes logs, shows file path and exits with message.

- 14) **FR-14. (System)** The System shall enforce turn alternation — players must alternate moves and the System shall refuse out-of-turn moves.

Acceptance: Any out-of-turn input is rejected with a clear message and re-prompt.

- 15) **FR-15. (System)** The System shall provide friendly, non-technical error messages for all input errors and will not crash on malformed input; it will re-prompt until valid input is received.

Acceptance: All invalid inputs produce single-line guidance (no stack traces) and allow retry.

- 16) **FR-16. (System)** The System shall track the SwiftBot's current location and orientation and use this to compute the motion commands necessary to reach each target square (uses calibrated seconds-per-25cm or equivalent parameter).

Acceptance: Motion planner parameters recorded in Implementation; motion commands reference current location/orientation.

Non-functional requirements

- 1) **NFR-01. (Interface)** The program shall use a command-line interface (CLI) for all textual input/output. The CLI shall display the ASCII game board, move summaries, prompts and errors.
- 2) **NFR-02. (Correctness)** All game logic (move validation, win/draw detection, score updates) shall be fully testable; unit tests should exist for these modules. *Measure:* 100% deterministic unit tests for rules.
- 3) **NFR-03. (Input validation)** The system shall validate all user inputs for format and range (e.g. [1,3] row/column) and reject anything outside 1..3 or non-numeric values. Error messages must be short and user-friendly.
- 4) **NFR-04. (Response time)** The CLI shall update the board and display move summaries within **1–2 seconds** of a confirmed move (excluding SwiftBot physical motion).
- 5) **NFR-05. (Logging)** The system shall reliably append a round log entry for every round; the log shall survive program crashes where possible (flush to disk after each round).
- 6) **NFR-06. (Modularity & Readability)** The codebase shall be modular (separate modules: input handling, game logic, motion command interface, logging) and include inline comments describing each public function's purpose.
- 7) **NFR-07. (Safety & Calibration)** The SwiftBot movement must be calibrated so that travel distances per square are accurate ($\pm$ acceptable tolerance). The environment must be well lit to observe LEDs/shapes. *Measure:* calibration routine verifies that motion for 25 cm is within $\pm 10\%$ before game start.
- 8) **NFR-08. (Robustness)** The system shall handle unexpected exceptions gracefully (log the exception, display a friendly message, and return to a safe state rather than crash).
- 9) **NFR-09. (Usability)** Error messages shall be concise and actionable; the CLI should show brief instructions for first-time users.
- 10) **NFR-10. (Testability)** The game logic should be independent of the SwiftBot hardware interface so it can be fully tested in a hardware-free mode (mock/swapped interface). *Acceptance:* tests run in CI without robot attached.

Additional functionalities

- 1) **AF-01. (Undo — limited) (User/System)** Provide an Undo feature: user may undo their last move up to two times per game. When undo is invoked via a CLI command (e.g. undo), the System reverts the last move (both board state and logged move), updates the scoreboard/state accordingly, and repositions SwiftBot to its pre-move location (motion command).
- 2) **AF-02. (Move Timer — configurable) (User/System)** Optional per-game timer: the user can enable a turn timer (configurable between **5–10 seconds**) before the first round. If enabled, the System starts a timer when prompting for the user's move; if the user does not submit a valid move before timeout, the turn is forfeited and control passes to the SwiftBot.
- 3) **AF-03. (Adaptive SwiftBot difficulty & movement style) (System)** Provide two SwiftBot move strategies selectable at game start: **Random** (default) and **Strategic** (minimax or heuristic). When Strategic is selected, the System computes the best board move and additionally causes SwiftBot to use a different LED pattern and slightly faster movement on decisive moves (visual feedback).

IMPROVEMENTS MADE TO THE INITIAL SRS

One of the improvements made to generated SRS was to keep the functional, non-functional requirements and additional functionalities atomic, clear and precise. The original SRS generated by the LLM covers all the functional, non-functional requirements and additional functionalities but each requirement is too complex and detailed, which would affect how the requirements are translated to the flowchart and pseudocode later. Keeping it atomic and simpler would make implementation a lot easier to follow and saves time.

Additionally, I had double-checked the generated srs to ensure that all requirements were met and ensured that the requirements followed a logical sequential order, which ensured that it makes it easier to follow step by step before planning the flowchart and pseudocode to reduce the chance making mistakes beforehand.

Finally, I used consistent numbering across my improved version to ensure that that each functional and non-functional requirements is atomic and easily searchable.

IMPROVED SOFTWARE REQUIREMENTS SPECIFICATION

1) INTRODUCTION

1.1) PURPOSE:

The **Noughts & Crosses (Tic-Tac-Toe)** game is designed to simulate a two-player game on a 3x3 grid, with one player being controlled by the user and the other by a robot (SwiftBot). The project will be implemented using a command-line interface (CLI), with the game logic running on a computer and the SwiftBot executing physical movements on the board based on the moves chosen by the player or the robot itself.

1.2) SCOPE:

- a) **Game Logic:** Handling player moves, win conditions, and game state.
- b) **User Interaction:** Providing an interface for the user to input their moves, view the board state, and interact with the system through the CLI.

- c) **SwiftBot Integration:** Enabling the robot (SwiftBot) to make moves, respond to user interactions, and show physical feedback (e.g., blinking LEDs for win/loss/draw).
- d) **Logging and Persistence:** Recording game results in a persistent log and maintaining a scoreboard to track performance over multiple rounds.
- e) **Calibration and Testing:** Ensuring that the SwiftBot's movements are accurate within a defined tolerance and calibrating the robot to fit the physical board's grid.

1.3) ACTORS:

- a) **User:** The person playing the game. They will enter their name, select moves on the 3x3 grid, and interact with the system through the CLI. The User controls the first player, and their actions determine the state of the game, including input validation and move confirmation.
- b) **SwiftBot (Robot):** A robot controlled by the system that makes random moves on the 3x3 grid when it is the second player's turn. The SwiftBot will move physically on the board, blink LEDs to indicate the state of the game (win/loss/draw) and return to its starting position after each move. It also performs basic calibration to ensure its movements are accurate.
- c) **System:** The software that controls the game logic, validates user input, updates the game board, maintains the scoreboard, and handles all aspects of the game from start to finish. The system includes the CLI interface, game state management, and the logic for interacting with the SwiftBot.

1.4) DEFINITIONS, ACRONYMS, AND ABBREVIATION:

- a) **SRS:** Software Requirements Specification
- b) **UI:** User Interface
- c) **CLI:** Command-Line Interface

1.5) SRS OVERVIEW:

This SRS outlines the functional and non-functional requirements for noughts & crosses, as well as outlining any additional functionalities that should be integrated alongside the required ones.

2) **FUNCTIONAL REQUIREMENTS**

2.1) PHYSICAL & ASCII GAMEBOARD DETAILS & GAME PIECES:

- a) a physical board for the swiftbot to move on
- b) The physical board will be a 3x3 (9 squares), an area of 75 by 75cm and will be numbered [1-3]. Each square will measure 25x25 cm.
- c) each player will have 4 game pieces each: X or O, to mark their chosen position.
- d) There will be a command-line ASCII representation of the 3x3 board digitally for the user to execute their moves, with clear row/column labels mapping out the coordinates of moves.
- e) All inputs and any outputs should be handled by the command-line interface.

2.2) ALIGNING BUTTONS WITH CORRESPONDING LED COLOURS:

- a) Button A (front left): Red
 - b) Button B (back left): Green
 - c) Button X (front right): Blue
 - d) Button Y (back right): Yellow
- 2.3) DISPLAY WELCOME SCREEN & USER REGISTRATION:
- a) The program will need to display a welcome screen with a user-readable message. It should have a brief description of the game rules, which must also be clear.
 - b) The program shall begin the registration sequence when the user presses the SwiftBot 'A' button.
 - c) the program should prompt the user to enter a username via the keyboard. The name should be in a string format; any other formats is invalid.
 - d) a name will be randomly generated and assigned to the swiftbot after the user enters a username which can be configured manually.
- 2.4) GENERATE & ROLL DICE:
- a) The program will roll a virtual 6-sided die for both the player and swiftbot automatically.
 - b) The player with the highest dice roll goes first and will be assigned the 'O' piece, whilst the other will have the 'X' piece.
 - c) if there's a tie, the system shall re-roll until a single higher value is produced and the starter is determined.
- 2.5) USER'S TURN:
- a) For the user's turn, the System shall prompt the User to enter a move in the format [row, column] within the range 1-3 in full integers.
 - b) The System shall validate and verify that the user input is numeric and in the right format.
 - c) If invalid, the System shall show an error and re-prompt.
 - d) The System shall check whether the selected square is empty
 - e) if a square is an unavailable, the System shall notify the User and re-prompt for a different move.
 - f) When the User provides a valid move, the System shall mark the internal board state with the User's piece ('O' or 'X') on the square.
 - g) The System shall ensure that the user cannot make consecutive moves
 - h) update the displayed ASCII board after the move is successful
 - i) displays the following message: '**[User – O/X] moved to square [r, c]**' after each successful move
- 2.6) SWIFTBOT'S TURN:
- a) When it's the swiftbot's turn, the robot will randomly select its move to its target square within the range
 - b) display the chosen square to the User *before* instructing the SwiftBot to move.
 - c) After going to the chosen square, the swiftbot will then execute a blink sequence in green 3 times upon arriving to the target square.

- d) The swiftbot will then return to its starting position in the board after the move.
 - e) the system should then display the move summary for swiftbot move:
[SwiftbotName – Piece] moved to square [r, c]
- 2.7) WIN/DRAW CONDITIONS, TRACING WINNING LINE & BLINKS LEDs:
 - a) If the user wins, the swiftbot will trace the winning line and blinks green 3 times before and after tracing for an 'O' win.
 - b) If the swiftbot wins, the swiftbot will trace the winning line and blinks red 3 times before and after tracing for an 'X' win.
 - c) If it's a draw, the swiftbot will spin once and blink in blue 3 times before and after the move.
- 2.8) MAINTAINING LOG FILE & UPDATE BOARD:
 - a) After each round (win or draw), the System shall append the following information in the log file: round number, the playing piece (i.e., 'O' or 'X'), the move positions (row and column), game result (win/draw), the winner player name or draw and current date & time.
 - b) The System shall update and maintain the scoreboard across rounds to track user performance
 - c) display the scoreboard results after each round.
 - d) When the user quits, the System shall close the log file and terminate the program
 - e) When the user or swiftbot wins a round, they will be awarded a score
- 2.9) PLAY AGAIN/QUIT OPTIONS:
 - a) **Button Y:** play another round and the system will reset the board and start a new game
 - b) **Button X:** quit and terminate the program

3) **NON-FUNCTIONAL REQUIREMENTS**

- 3.1) USABILITY:
 - a) The game interface will be implemented as a simple command line interface console that is easy to navigate and intuitive.
 - b) Console UI formatting and design structure should be clear, simple and organised, displaying clear information.
 - c) Error messages should be clear, concise and on to the point. Non-technical users should not see that confusing paragraph/block of error message.
 - d) Displays game board in console game in clear and precise format.
- 3.2) PERFORMANCE
 - a) The system response time should be shorter from 1-2 seconds for updating player moves, calculating positions, updating boards.
- 3.3) RELIABILITY
 - a) There will be effective and reliable input validation to check the format, data type and numeric ranges for user input.

- b) Efficient error-handling to manage exceptions, prevent unexpected outcomes like crashes and bugs and handle it gracefully.
- 3.4) PORTABILITY
 - a) The program should run on multiple platforms that is java-compatible and supports java runtime environment
- 3.5) MAINTAINABILITY
 - a) Code comments should be provided to explain each method's functionalities and purpose.
 - b) Code should be modular, which each block of code representing a specialised method performing a single function
- 3.6) SCALABILITY
 - a) The system should only have two players – the user and the swiftbot, as this is a two-player game.
 - b) A log file that consistently tracks and records game events and actions like player names, scoring, player positions, movements, wins, losses and draws.
- 3.7) SECURITY
 - a) the game doesn't handle sensitive data and therefore does not need any security strategies and backups to handle such.
- 3.8) USER INTERFACE
 - a) All inputs and outputs will be handled via the command-line console window
 - b) The user will interact with the system via the swiftbot buttons and the physical keyboard.
 - c) The system will handle swiftbot movements during each round
 - d) Swiftbot LEDs will light up depending on the button pressed, for example, pressing button 'A' will activate the red LED.
 - e) The user is expected to input their moves in an integer format and should be within the appropriate range of the board.

4) ADDITIONAL FUNCTIONALITIES

- 4.1) Choice to undo the last move the player makes but has a limit of 2 times (including the players first move) to prevent them from too easily gaining an advantage and allow them to think quicker and more strategically to balance out the odds of the game.
- 4.2) a timer will be set where the user will have to put their piece on the game board and will allow the player to adjust from 5-10 seconds the time the program will recognise the validity of the move. After that time, the move will become invalid and will give that turn to the next player.
- 4.3) The game will have a difficulty settings option at the start of the game, in the introduction, where the command line interface will display options, which are 'easy', 'medium' and 'hard' after the introduction of the game, asking the user '*what game mode? Enter easy, medium or hard*'. the difficulty of the game will depend on the two functionalities stated above:

- a) How much **'undo's'** the user will have depending on game difficulty. For instance, the 'easy' mode will have 2 new move undo's, meaning the user can revise their move choices back by removing their piece twice. Whereas the medium and hard modes will have 1 and none respectively.
- b) This will prompt the program to ask, **'do you want to undo your last move, enter yes or no?'**, if yes, the gameboard will remove your last piece to start again or if no, then the pieces stays there.
- c) The program will keep track of the number of times the user uses the undo option based on their difficulty. This value will be enclosed in brackets like **'Undo move chances: 2'** after the program asks yes or no. this will be shown in the command line as **'do you want to undo your last move, enter yes or no? (Undo move chances: 2)'**,
- d) For each turn, a timer countdown will be enforced as a time limit the user is allowed during their move. For the easy mode, there won't be a timer but a time limit of 10 seconds for the medium mode and 5 seconds for the hard mode will be applied.

2. ALGORITHM DESIGN

Name of LLM used: OpenAI ChatGPT & Anthropic Claude

Model version: GPT-5.1 mini & Sonnet 4.5

Access method: ChatGPT web (free tier) & Claude Web

Date & time accessed (GPT-5.1 mini) to generate the flowchart based on the LLM-generated srs: 19 December 2025, 21:00

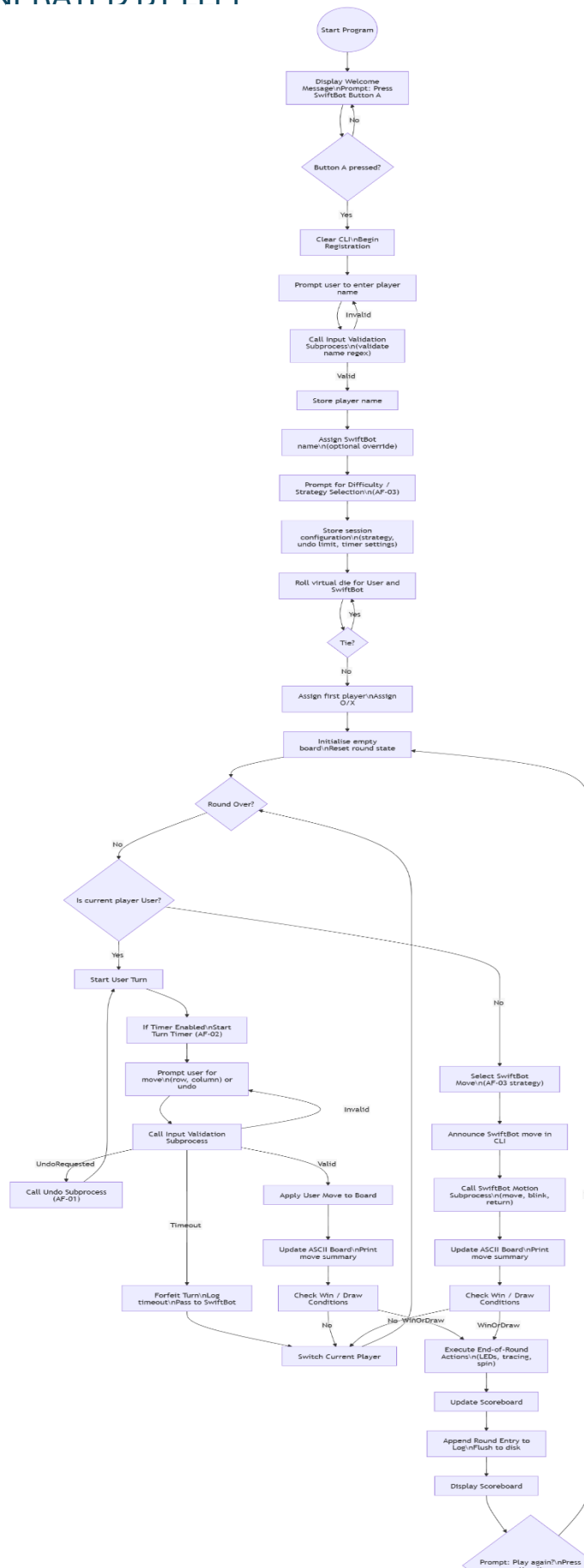
Date & time accessed (Claude Sonnet 4.5) to generate the improved flowchart based on the improved srs: 3 February 2026, 20:40

Prompts used

- 1) **Based on the LLM-generated SRS:** *'based on this srs above, can you generate a main flowchart via mermaid code'*
- 2) **Based on the improved SRS (Claude Sonnet 4.5):** *'based on week 4 and 5, and the pasted text uploaded above, can you edit and ensure the existing flowchart generated so that it follows flowchart conventions like correct shapes, processes, and data flow for each step (e.g, decision diamond for a yes/no) and can you separate this main flowchart into 7 integrated subprocesses flowcharts where each subprocesses represents the: 2.3) DISPLAY WELCOME SCREEN & USER REGISTRATION, 2.4) GENERATE & ROLL DICE, 2.5) USER'S TURN, 2.6) SWIFTBOT'S TURN, 2.7) WIN/DRAW CONDITIONS, TRACING WINNING LINE & BLINKS LEDs, 2.8) MAINTAINING LOG FILE & UPDATE BOARD and 2.9) PLAY AGAIN/QUIT OPTIONS with the other two functional requirements, and the rest of the SRS being fully integrated into the main flowcharts and its 7 subprocesses.'* remember to keep the flowcharts simple as possible whilst following the SRS
- 3) **Converting it into mermaid code request:** *'can you do this all in mermaid code but let me convert it online from mermaid'*

NOTE: for the improved flowcharts, I used my improved SRS to generate my improved versions and then I manually checked and improved on each step of the designs against the SRS to ensure that the correct notations, shapes and messages are used, and the logical flow is correct.

FLOWCHART GENERATED BY IIM



LLM-GENERATED FLOWCHART CRITIQUE

Flowchart shapes: some of the flowchart shapes used for each process are wrong. For instance, for the start and end terminals, the shapes cannot be a circle or a rectangle but should be an oval or a shape with 2 semicircular ends.

Arrow lines: there are two arrows points upwards and downwards between transitions of activities rather than a single linear downwards arrow representing data flow. For example, the transition between where the flowchart displays the welcome message to asking the user if they pressed button 'A', showing two arrows flowing at opposite directions, which is not allowed.

Decision diamonds: some decision diamonds don't have a clear 'yes' or 'no' pathway; some only have a 'no', making it confusing and inconsistent.

Lack of separate subprocess flowcharts: based on the functional and non-functional requirements, there're no separate flowcharts for each subprocess. This makes the main flowchart much harder to interpret and understand. There're also no flowchart symbols showing each subprocess.

Lines crossing each other: there are lines that intersects each other such as the 'win/draw' indicator. This is not allowed and makes the logical flow much harder to trace.

Unclear text: the text displayed in the flowchart shapes are unclear, vague or too short. For instance, the welcome message at the start says, 'Display Welcome Message\prompt: PressSwiftBot Button A'.

Too many steps: the flowchart is currently too complex and contains too much steps rather than containing much of these steps within each subprocesses.

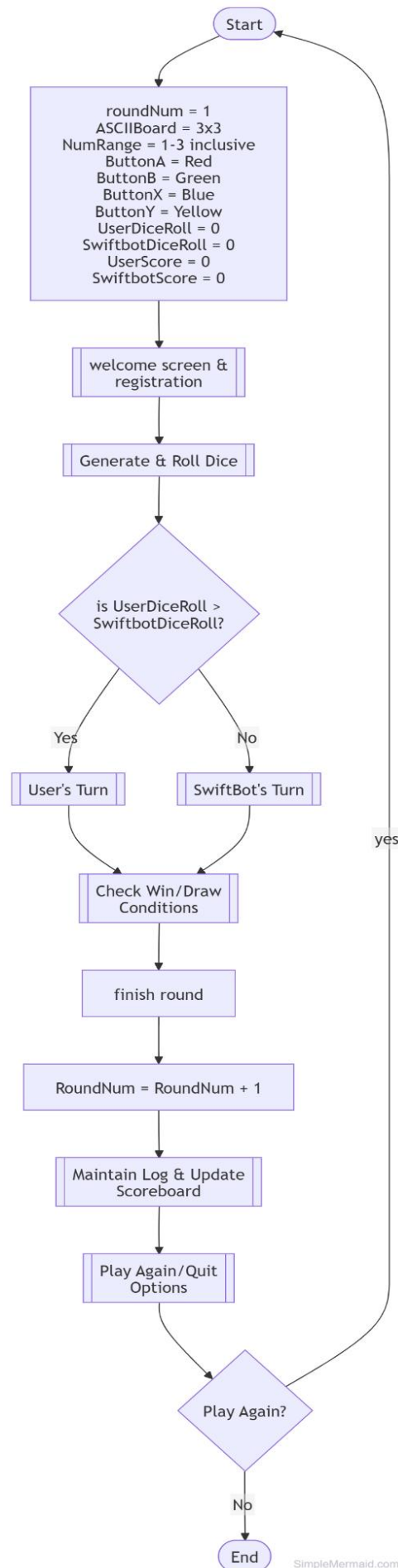
HOW I IMPROVED ON THE LLM-GENERATED FLOWCHART

Using my improved srs in ChatGPT to generate my improved flowcharts, I then manually checked and edited parts of the process the improved main flowchart and its subprocesses generated to ensure that it aligns with the improved SRS's functional, non-functional requirements and additional functionalities.

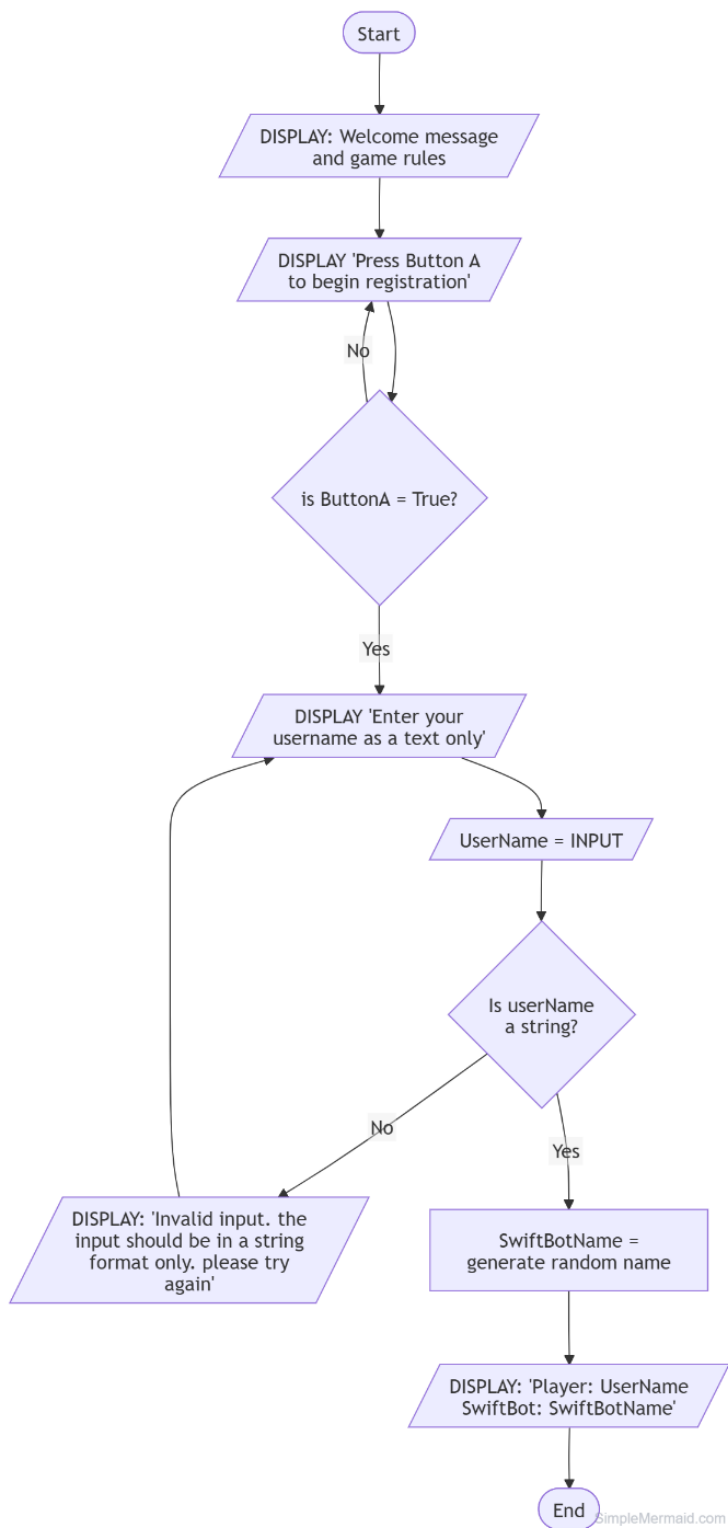
- 1) **Flowchart shapes:** I checked every single part of the LLM-generated flowchart and fixed some of the shapes generated are incorrect or missing such as using a rectangle instead of a parallelogram for user inputs an important note is that the AI sometimes generated the end terminals as a full circle which is incorrect so I will need to correct that. this is also true for other the shapes in the flowcharts as well. For some of the workflows, a shape representing a subprocess is needed as this encapsules the workflow.
- 2) **Simplified the main flowchart:** the original llm-generated flowchart was too complex and nearly unreadable (because it's too long) due to the lack of modularity of the flowchart into subprocesses. Therefore, I tell the LLM to generate another flowchart but split it into subprocesses, and it generated 9 subprocesses based on reading the improved srs and evaluating the complexity of the requirements.
- 3) The new main flowchart is **clean, orderly and follows a logical path** without any lines crossing across each other and unnecessary complexity

- 4) To ensure to improve **readability and visibility**, I used a different mermaid-to-flowchart converter website called mermaid.ai as it allows more manually flexibility in adjusting the flowchart shapes, arrows and text, allowing me to make it more visually intuitive.

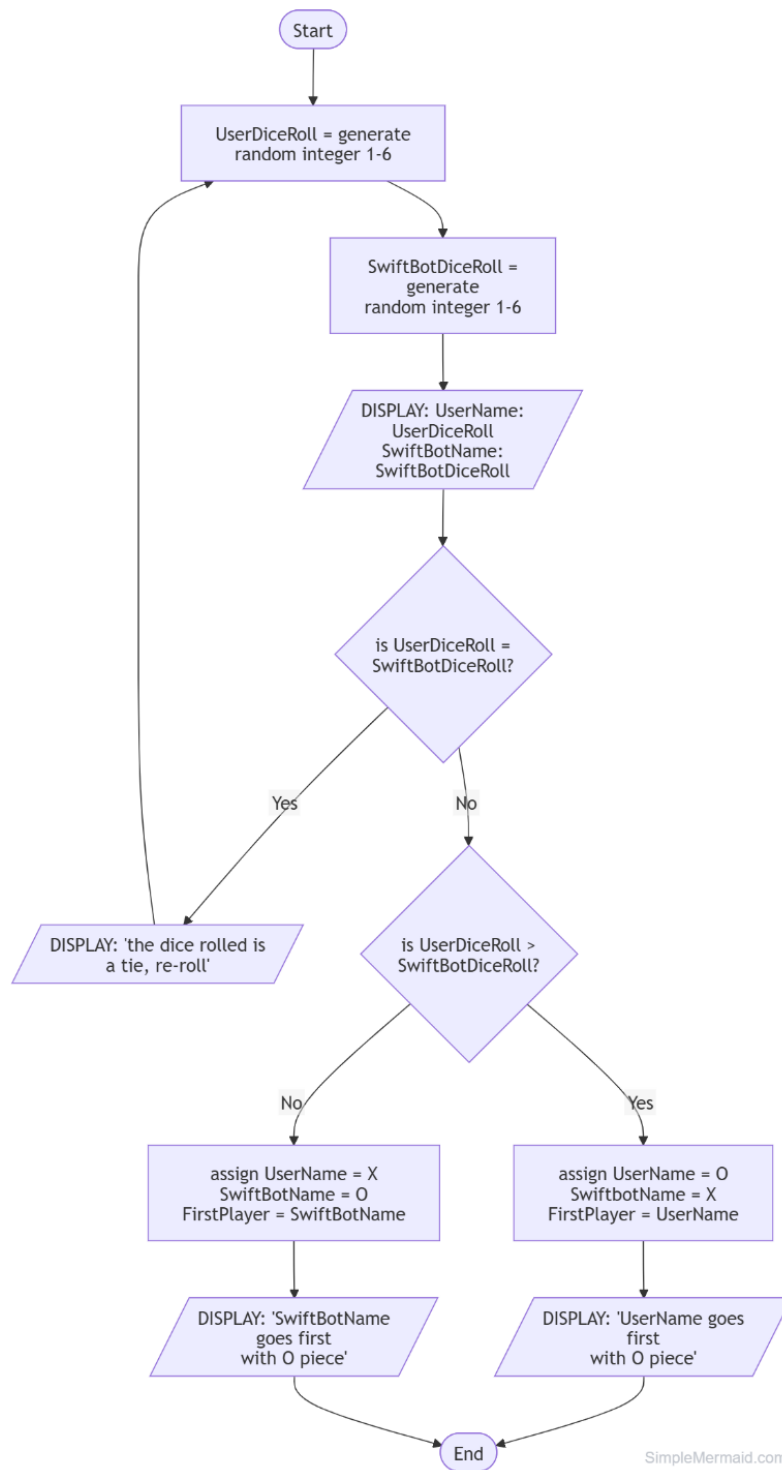
Improved main flowchart



INITIALIZATION AND REGISTRATION

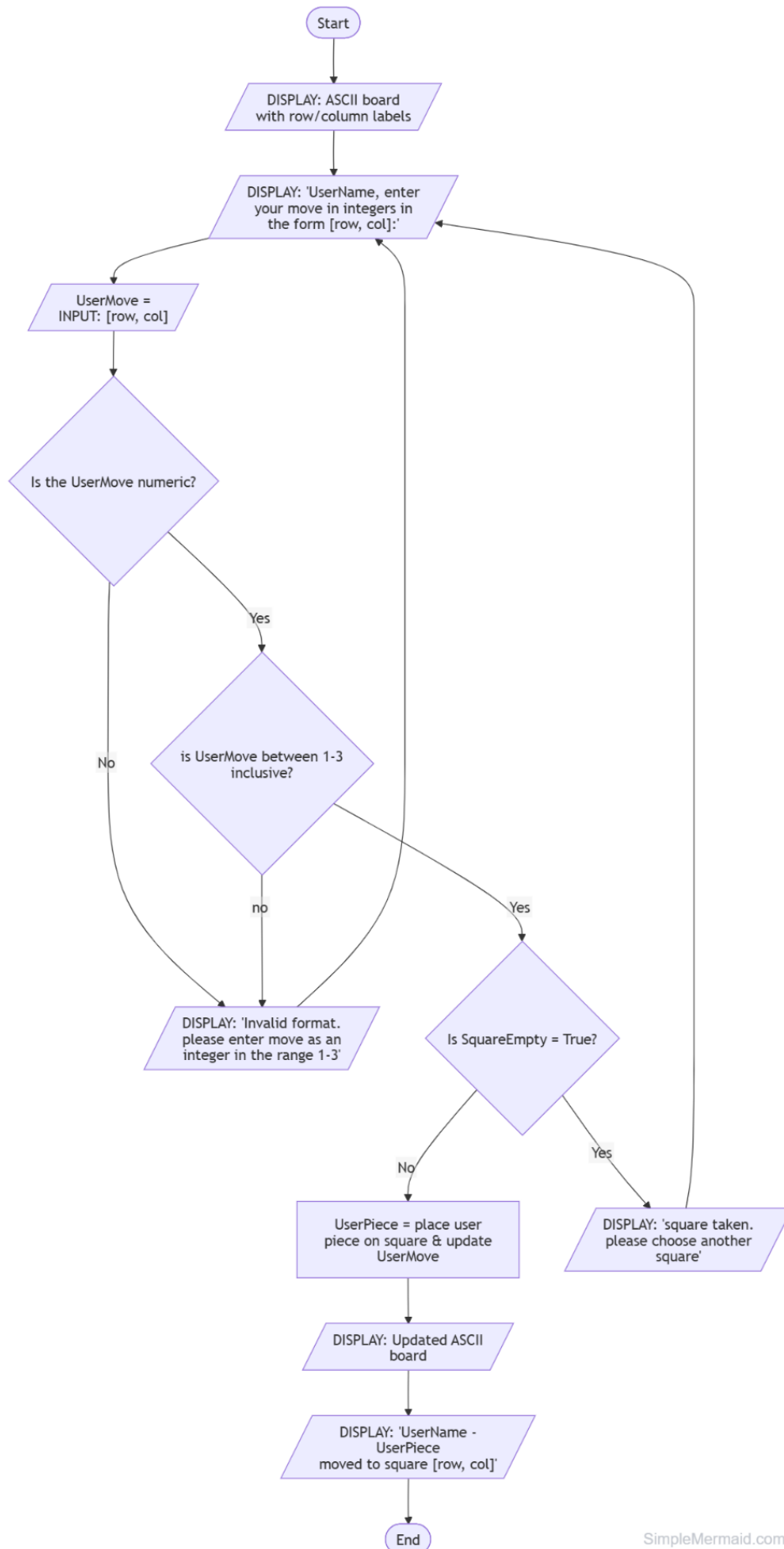


GENERATE & ROLL DICE

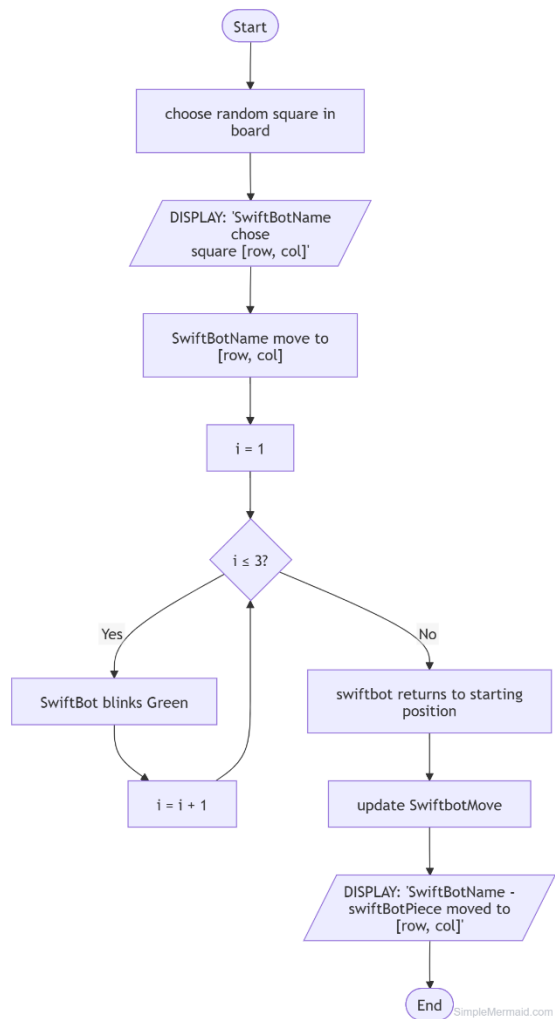


SimpleMermaid.com

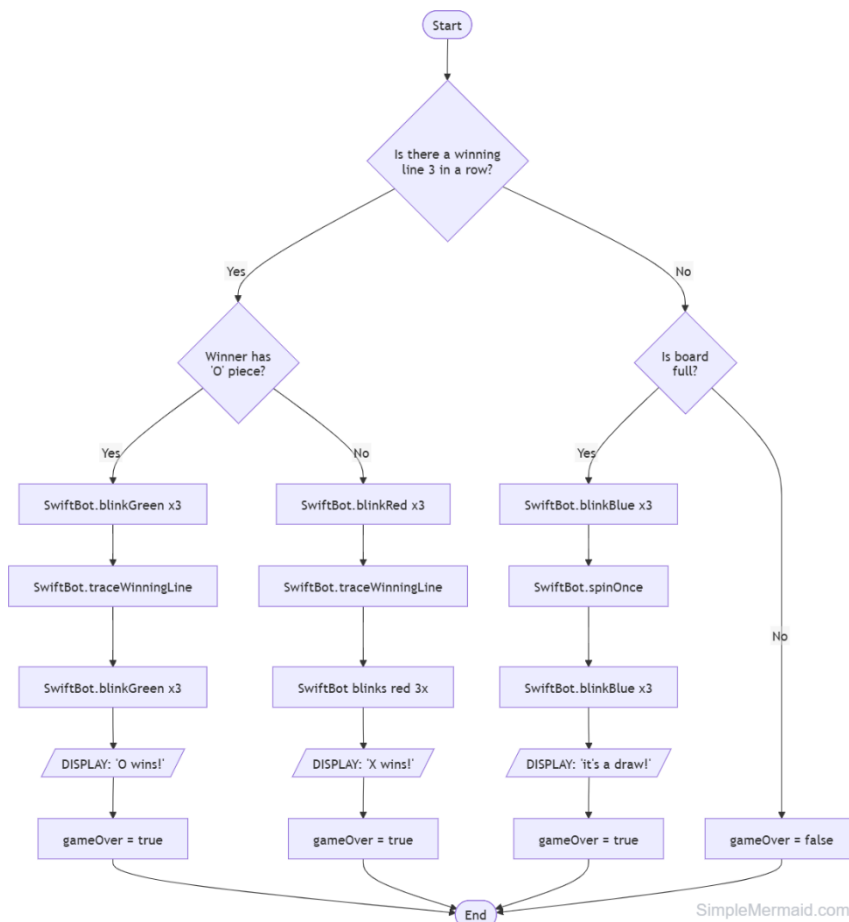
USER'S TURN



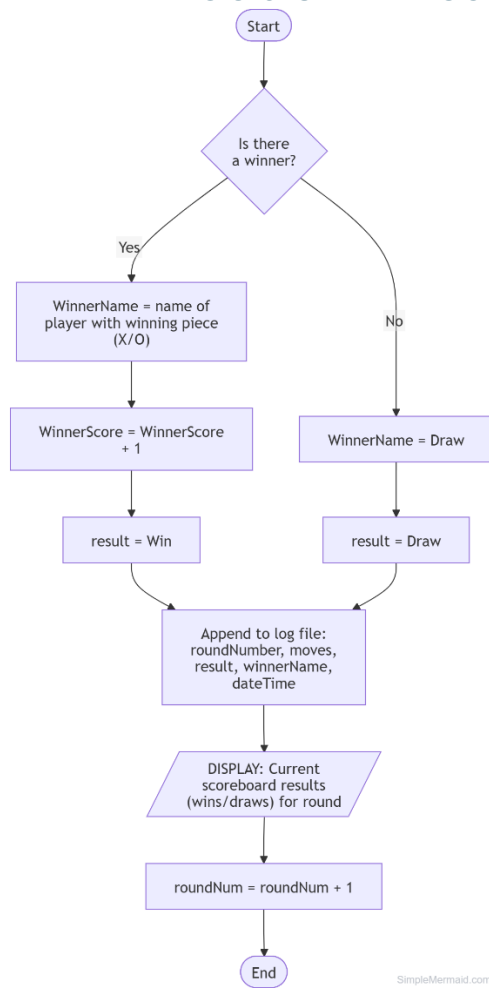
SWIFTBOT'S TURN



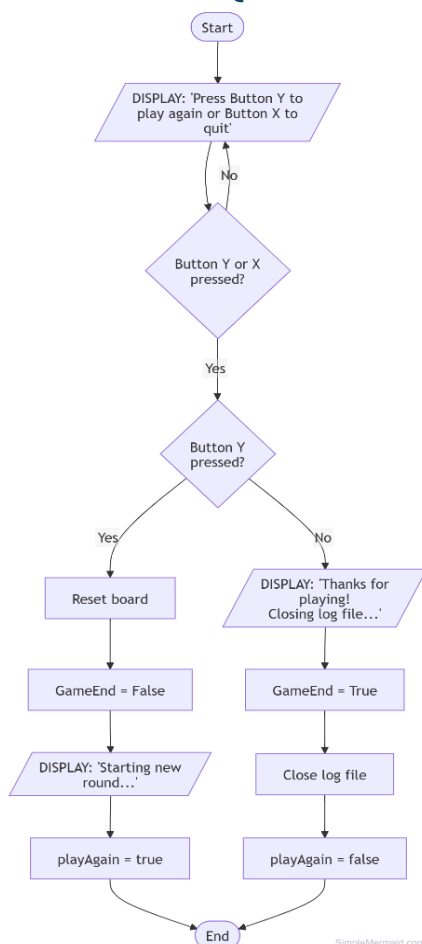
CHECK WIN/DRAW CONDITIONS & BLINK LED



MAINTAIN LOG & UPDATE SCOREBOARD



PLAY AGAIN/QUIT OPTIONS



USER INTERFACE DESIGN

Name of LLM used: Anthropic Claude

Model version: Sonnet 4.5

Access method: Claude web (free tier)

Date and time accessed to generate UI designs: 4 February 2026, 18:47 UTC

Prompts used

To generated UI designs based on the llm-generated srs and flowcharts (ChatGPT-5.1 mini): 'based on the LLM-generated srs at the top and the LLM-generated flowchart in the image, can you generate an single initial UI design as a windows command-line environment design and should illustrate how interactions would appear as if they were in the actual windows command line interface'

LLM-generated UI designs

1) Welcome / Start (FR-01, NFR-01, NFR-09)

markdown

```
Microsoft Windows [Version 10.0]
(c) 2026 SwiftBot Labs. All rights reserved.

=====
      NOUGHTS & CROSSES  - SwiftBot (CLI)
=====

Welcome! Play Noughts & Crosses (3x3) against SwiftBot.

Short rules:
- Enter moves as row,column using numbers 1..3 (example: 2,3)
- Difficulty sets Undo quota & Turn timer
- Use 'undo' to revert your last move (if available)
- Hardware buttons also accepted (A/B/X/Y) - or use keyboard

Press SwiftBot Button A (or type 'A' + Enter) to begin...
>
```

3) Difficulty & Session config (AF-03, AF-01/AF-02 config)

pgsql

```
--- Game Setup: Difficulty & AI Strategy ---
Choose difficulty (type): easy / medium / hard
> medium

Selected: MEDIUM
• Undo quota: 1 per game
• Turn timer: 10 seconds
• SwiftBot strategy: RANDOM (default)

Confirm settings? (Y/N)
> Y

[OK] Configuration saved: undo_quota=1, turn_timer_seconds=10, ai_strategy=RANDOM
(Press Enter to roll dice and choose starter)
>
```

2) Registration — Player name & SwiftBot name (FR-02, FR-03)

pgsql

```
--- Registration ---
Enter your player name (alphanumeric, 1-30 chars):
> Yasoda

[OK] Player name saved: Yasoda

SwiftBot default name: "Robo-42"
Override SwiftBot name? (Y/N)
> N

[OK] SwiftBot name: Robo-42
(Press Enter to continue)
>
```

4) Dice roll to choose starter (FR-04)

csharp

```
Rolling virtual die to choose who goes first...
[Yasoda] rolled: 4
[Robo-42] rolled: 6

Result: Robo-42 goes first and will play '0'
(Press Enter to start round)
>
```

5) Main board display (initial & after each move) — ASCII board (FR-09)

```
less

      Columns ->   1   2   3
Rows
  1           [ ] [ ] [ ]
  2           [ ] [ ] [ ]
  3           [ ] [ ] [ ]

Move summary: (none yet)
Current turn: Robo-42 (0)

(If you are the User, enter move row,column OR type 'undo')
Undo moves left: 1    Time left: [00:10]

>
```

After a move (example)

```
less

      Columns ->   1   2   3
Rows
  1           [ ] [X] [ ]
  2           [0] [ ] [ ]
  3           [ ] [ ] [ ]

Move summary: [Robo-42 - 0] moved to square [2,1]
Current turn: Yasoda (X)
Your move (format 2,3 or 'undo') >
```

7) Undo interaction (AF-01)

```
pgsql

> undo
Undo moves remaining: 1
Confirm undo last move? (Y/N)
> Y

[OK] Last move undone. Undo moves left: 0
[LOG] UNDO appended to round log (original move preserved; UNDO annotated)
Board updated.
```

If no undo left

```
markdown

> undo
Undo moves remaining: 0
Cannot undo further moves.

>
```

6) User Move Prompt, Validation & Error messages (FR-05, FR-06, FR-103)

```
pgsql

Your move (type 'row,column' e.g. 2,3) or 'undo'
Undo moves left: 1    Time left: [00:08]
> 2,1

[OK] Yasoda (X) moved to square [2,1]
(ASCII board updated)
```

Invalid input examples

```
pgsql

> two,one
Invalid input. Expected format row,column where each is 1..3. Try again.
>

> 4,1
Invalid input. Row/Column must be 1,2 or 3. Try again.
>

> 2,1
Square [2,1] already occupied by [0]. Choose another square.
>
```

Out-of-turn

```
pgsql

> 3,2
Input rejected: Not your turn. Wait for the current player to move.
```

8) Move timer behavior (AF-02)

Countdown visual (updates each second in CLI)

```
less

Your move. You have 10 seconds.
Time left: [00:10] > two,two
Invalid input. Format: row,column. Time left: [00:07]
Try again:
Time left: [00:05]
Time left: [00:04]
Time left: [00:03]
Time left: [00:02]
Time left: [00:01]
*** TIME EXPIRED — turn forfeited ***
[LOG] FORFEIT recorded for Yasoda at 2026-02-05T14:12:03Z
(Robo-42 will now make a move)
```

9) SwiftBot decision, announcement & motion logs (FR-07, FR-08, FR-16)

```
scss

[AI] Robo-42 (0) will move to square [3,3] <-- announced BEFORE motion
(Press Enter to execute movement or wait)
>

[Motion] Command: move -> cell [3,3]
[Motion] Pre-move LED: GREEN x3
[Motion] Traveling to [3,3]... (eta ~2.1s)
[Motion] Reached [3,3]; Post-move LED: GREEN x3
[Motion] Returned to starting pose
[LOG] Motion appended: start_pose, end_pose, duration, timestamp
Move summary: [Robo-42 - 0] moved to square [3,3]
```

Calibration fail example

```
pgsql

[ERROR] Motion refused – calibration out of bounds (measured ±12% > allowed ±10%).
Action: Motion aborted. Please run 'calibrate' (see Appendix).
(Technical details logged: telemetry_2026-02-05.log)
```

10) Win / Draw detection & tracing (FR-10, FR-21..FR-24)

User wins (O):

```
yaml

*** WINNER: Yasoda (O) ***
[Motion] SwiftBot blinks GREEN x3 (pre-trace)
[Motion] SwiftBot tracing line: [1,1] -> [1,2] -> [1,3]
[Motion] SwiftBot blinks GREEN x3 (post-trace)
Round recorded at 2026-02-05T14:20:21Z
```

SwiftBot wins (X):

```
java

*** WINNER: Robo-42 (X) ***
[Motion] SwiftBot blinks RED x3 (pre-trace)
[Motion] SwiftBot tracing line ...
[Motion] SwiftBot blinks RED x3 (post-trace)
Round recorded.
```

Draw:

```
csharp

*** ROUND RESULT: DRAW ***
[Motion] SwiftBot spins once and blinks BLUE x3.
Round recorded.
```

11) End-of-round, scoreboard & play-again prompt (FR-11, FR-12, FR-13)

```
yaml

--- Round summary ---
Round #: 2
Moves: Yasoda (X) [1,1], Robo-42 (O) [1,2], ...
Result: Yasoda wins
Timestamp: 2026-02-05T14:20:21Z

Scoreboard:
Yasoda: 2
Robo-42: 0

Play again? (Press SwiftBot Button Y or type 'Y' -> play another round)
Or press SwiftBot Button X or type 'X' to quit
>
```

If Quit selected

```
yaml

Flushing logs to disk...
[OK] Log file saved to: C:\Users\you\Documents\swiftbot_logs\noughts_log_2026-02-05_14202
Final scoreboard:
Yasoda: 2
Robo-42: 0

Program terminating. Goodbye.
```

Improved UI prototype designs

1) WELCOME SCREEN

Welcome/start screen

NOUGHTS & CROSSES

hello, let's play noughts & crosses against swiftbot

GAME RULES

-
- 1) Enter moves only in the format [row, column], these should be integers, no other formats is acceptable.
 - 2) There will be 3 game difficulty modes: easy/medium/hard
 - 3) The easy mode won't have a time limit countdown and will have 2 undo's maximum in which a user can revise back a move choice if they don't like.
 - 4) A time limit for the medium and hard modes are there to ensure that the user thinks quickly within that time limit before their move is expired and passed on to the swiftbot.
 - 5) The medium and hard modes of the game is much stricter with time pressure and undo limits. (Medium: time limit of 10 seconds and 1 undo's, hard: 5 seconds with no undo's).
 - 6) The game is won when the player gets 3 'X' or 'O' in a row, whether horizontally, diagonally and vertically, otherwise, the swiftbot wins. (swiftbot blinks green 3 times before and after tracing in the shape of an 'O' and red for 'X' win)
 - 7) If it's a tie, the swiftbot will blink its blue lights 3 times before and after performing a spin once.

If your ready after reading the rules, press button 'A' on the swiftbot to begin:

>

2) USER REGISTRATION

User registration screen

REGISTRATION

Now, can you please enter a username please (only letters, numbers and symbols are allowed in a single string format, no spaces, capitals or other data types are permitted) Maximum username length (15 characters):

>albert77

Registering username.....

[SUCCESS] your username has successfully been saved, *albert77*

Generating swiftbot username.....

The swiftbot has now been assigned the username '*robo23*', do you want to change it? enter 'yes' to change and 'no' to remain the same.

>no

The swiftbot name will now remain '*robo23*'

For invalid inputs:

Now, can you please enter a username please (only letters, numbers and symbols are allowed in a single string format, no spaces, capitals or other data types are permitted):

>albert 77GJIS

[ERROR] your username cannot contain spaces, capitals or any other data type other than string and should not exceed the maximum length of 15 characters, please try again:

>

3) ROLL DICE & GAME DIFFICULTY

Game difficulty and roll dice

Main game board (enter and undo moves)

	ROW
COLUMN	[] [] []
	[] [] []
	[] [] []

Can you please enter a move in the form [row, column] in the range 1-3 inclusively?

Enter a row number:

> 3

Enter a column number:

> 2

Processing move.....

[SUCCESS] your move has successfully been processed, do want to undo your move, if yes, enter 'undo'

[current number of undo's: 1]:

> undo

Undo move.....

[SUCCESS] your move has been undone, please enter your move again [10 seconds to move]:

Time left: 00:10

Time left: 00:09

Time left: 00:08

Now press 'Enter' to begin round:

> [ENTER]

Enter a row number:

> 1

Enter a column number:

> 2

Processing move.....

[SUCCESS] your move has successfully been processed [current number of undo's: 0]

The swiftbot will now choose it move.....

The swiftbot moved to the square [2][2]

Enter a row number:

>

Continued.....

5) SWIFTBOT BLINK LIGHTS

Swiftbot blinking lights three times before and after tracing line for wins/draws (user & swiftbot)

WINNER: *albert77*

[SWIFTBOT MOVING] blinks in GREEN 3 times.....

[MOTION] swiftbot tracing.....

[SWIFTBOT MOVING] blinks in GREEN 3 times.....

Logging the current round.....

Round successfully logged – *albert77*[O] - winner

WINNER: *robo23*

[SWIFTBOT MOVING] blinks in RED 3 times.....

[MOTION] swiftbot tracing.....

[SWIFTBOT MOVING] blinks in RED 3 times.....

Logging the current round.....

Round successfully logged – *robo23*[X] - winner

Result: DRAW

[MOTION] swiftbot spins once and blinks in BLUE 3 times

Logging the current round.....

Round successfully logged – *albert77*[O] & *robo23*[X] – draw

6) INVALID & OUT OF RANGE MOVES

invalid move

Enter a row number:

> one

[ERROR] the input isn't valid, please enter an integer. No other data types are allowed.

Enter a column number:

> two

[ERROR] the input isn't valid, please enter an integer. No other data types are allowed.

Out of range

Enter a row number:

> 4

[ERROR] that number is out of range. Please enter within the range 1-3 inclusive

Enter a column number:

> 6

[ERROR] that number is out of range. Please enter within the range 1-3 inclusive

7) OCCUPIED SQUARES & WAITING FOR SOMEONE'S TURN

Occupied square

Enter a row number:

> 3

Enter a column number:

> 2

[WARNING] the square you're moving to is occupied, please try another move:

Enter a row number:

>

Can't move during someone else's turn

Enter a row number:

> 3

Enter a column number:

> 2

[WARNING] your move is rejected, the swiftbot is current moving, please wait until they finish

8) END OF ROUND SUMMARY, SCOREBOARD AND PLAY AGAIN

End of round summary, scoreboard and play again

ROUND SUMMARY

ROUND COUNT: 6

Winner: *albert77*, congrats, you won!

Moves: [*3*, *2*], [*3*, *1*], etc – *albert77*

Moves: [*1*, *2*], [*2*, *3*], etc – *robo23*

SCOREBOARD:

Albert77: 4

Robo23: 2

Play another round? Press button 'Y' to play again or 'X' to quit

> quit

Saving round results.....

Saved results in the log to: *C:\Users\albert\Documents\logs\noughts & crosses.log*

----- See you next time -----

HOW I IMPROVED ON THE EXISTING GENERATED UI DESIGNS

- 1) **Made sentences clearer and easier to explain** – I made generated explanations clearer and easier to understand by ensuring that it guides the user without the confusion of shorter sentences that could de-track user attention.
- 2) **Used consistent formatting and colour schemes** – by applying an all-black or white tone for all messages and symbols consistent for general purpose system functions, I minimize the risk of distractions, and confusions, especially for users that are colour blind or other visual impairments. Instead, the only colours I've applied to are the user and swiftbot names, as well as the dice roll numbers generated. This two-colour contrast makes the overall interface smoother and comfortable.
- 3) **Makes error or non-error messages clearer and tells the user what to do next without overtly quitting automatically**– at the beginning of the messages, a word in capitals in squared boxed notation signifies what the message is. For example, an error showing a number is out of range will appear as this: '[ERROR] this number is out of range'. The first word in bold makes it easily identifiable as an error without confusion. the messages are also re-structured in the sense that they accommodate warnings for the user inputs to prevent a wrong action and tells them exactly what the issue is and re-prompt without abruptly terminating the program.

PLANNING AND MONITORING PROGRESS

What have I planned since then?

I had used GitHub classroom version to store all my report files, images and diagrams and potentially code in a secure, private repository so that that I can access it in future whether I needed without being lost.

At first, my commits have been relatively irregular and inconsistent due to focusing too much on one section and then completing another without fully completing or refining it. however, after I started to go back to my sections one by one, learning more about how GitHub works beyond git-bash cmd alternatives (such as uploading files manually) and receiving feedback from my group leader, I started to incrementally commit per new refinement/task I'm doing.

For the SRS, flowchart and UI design (both llm-generated and improved), I commit one message during intervals of the design phase without committing all work straight at once, as this approach suggest bad planning, monitoring skills and time management, as with the mistakes I made earlier.

Creating a Gantt chart

A Gantt chart here helps me more accurately track my progress of each incremental task by setting a weekly interval that covers both the design and implementation phases. For the design phase, for example, the colour codes show which tasks have been completed from last time to recent, where different colours show varying levels of completion in %. (greenest: 100%, reddest: 0%). I marked dates after February 2-8th with a cross to show the dates i will be implementing each task or feature on certain dates to avoid doing multiple tasks on the same time interval, or at least it makes room to implement other activities.

GANTT CHART FOR TRACKING/MONITORING PROGRESS & PLANNING FOR SOFTWARE IMPLEMENTATION IN ASSIGNMENT 2 & 3

STAGE	ACTIVITY	PROGRESS (%)	Dec 8 th – 14 th	Dec 15 th – 21 st	Dec 22 nd – 28 th	Dec 29 th – Jan 4 th	Jan 5 th – 11 th	Jan 12 th – 18 th	Jan 19 th – 25 th	Jan 26 th – Feb 1 st	Feb 2 nd – 8 th	Feb 9 th – 15 th	Feb 16 th – 22 nd	Feb 23 rd – Mar 1 st	Mar 2 nd – 8 th	Mar 9 th – 15 th	Mar 16 th – 20 th
SOFTWARE DESIGN	Generate initial SRS by LLM by choice	100 %															
	Improvements made to the generated SRS	100 %															
	Generate flowcharts via LLM based on the generated SRS	100 %															
	Improve generated flowcharts based on the improved SRS	90%															
	Divide the flowchart into subprocesses	100 %															
	Generate initial UI designs based on the generated SRS and flowcharts	100 %															
	Improve on the initial UI designs based on the improved SRS & flowcharts	90%															
	Review and refine improvements by gaining feedback from others continuously	50%															
	Assignment 2 final submission	95%															

SOFTWARE IMPLEMENTATION	Have a integrated development environment (IDE) such as eclipse, IntelliJ and vscode in place and ready.											X					
	Implement event handling of key presses: swiftbot button A upon starting user registration, etc												X				
	Username registration and game initialization												X				
	Input validation, verification and error handling													X			
	Managing user/swiftbot turns & moves														X		
	Gameboard set-up and handling														X		
	Handle dice roll loop logic														X		

Swiftbot motion & calibration															X		
Managing and update log file															X		
Handle LED blinking behaviour															X		
Black box testing using test tables																X	
Debugging																X	
Implementation report																	X
source code packaging																	X
Assignment 3 final submission																	X

LEGEND:

- COLOR-CODED (GREEN, BLUE, ETC) – means completed/partially completed
- CROSS – expected/planned future milestone
- BLANK CELL – no activity

REFERENCES

- 1) Mermaid.ai. *main flowchart*. Available on: <https://mermaid.ai/app/projects/1fbbbb1b-357c-4dbc-85f0-1cbff9f1b005/diagrams/8805d746-b5a8-4a66-99df-133daeb39ea1/version/v0.1/edit> (accessed: 20 January 2025)
- 2) SimpleMermaid.com. Available on: <https://www.simplemermaid.com/mermaid-tool.html> (accessed: 18 January 2025)
- 3) ChatGPT.com. *SRS Flowchart Breakdown*. Available on: <https://chatgpt.com/c/69651c10-2de0-832d-9ceb-9350dc269c44> (accessed: 16 January 2025)
- 4) ChatGPT.com. *noughts and crosses*. Available on: <https://chatgpt.com/c/693708ca-c110-832c-a638-a89209a49c35> (accessed: 18 January 2025)
- 5) Claude – *Flowchart redesign with subprocess separation*. Available on: <https://claude.ai/chat/38616e7c-cfc9-4121-ad9a-da89eb885c38> (accessed: 04 February 2026)